



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ДОНСКОЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(ДГТУ)**

Факультет «Информатика и вычислительная техника»

Кафедра «Кибербезопасность информационных систем»

Зав. кафедрой «КБИС»

подпись О.А. Сафарьян
«__» _____ 2025 г.

КУРСОВАЯ РАБОТА

Тема: «МОДЕЛЬ ПОМЕХОУСТОЙЧИВОГО КАНАЛА СВЯЗИ НА ОСНОВЕ БЧХ-КОДОВ»

Дисциплина: «Теория кодирования, сжатия и восстановления информации»

Специальность: 10.05.01 Компьютерная безопасность

Специализация: Математические методы защиты информации

Обозначение курсовой работы ТКСВИ.140000.000 Группа ВКБ42

Обучающийся _____ И. А. Ковалец
подпись, дата

Курсовая работа защищена с оценкой _____

Руководитель работы _____ доцент, Р. В. Егорова
подпись, дата

Ростов-на-Дону
2025



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ДОНСКОЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(ДГТУ)**

Факультет «Информатика и вычислительная техника»

Кафедра «Кибербезопасность информационных систем»

Зав. кафедрой «КБИС»

подпись О.А. Сафарьян
«__» _____ 2025 г.

ЗАДАНИЕ

на выполнение курсовой работы

Тема «МОДЕЛЬ ПОМЕХОУСТОЙЧИВОГО КАНАЛА СВЯЗИ НА ОСНОВЕ БЧХ-КОДОВ»

Дисциплина: Теория кодирования, сжатия и восстановления информации

Обучающийся: Ковалец Илья Андреевич

Обозначение курсовой работы ТКСВИ.140000.000 Группа: ВКБ42

Срок представления работы к защите «__» _____ 2025 г.

Исходные данные для курсовой работы:

1. Задание на выполнение курсовой работы;

Содержание пояснительной записки

Введение:

В данной курсовой работе рассматриваются учебные помехоустойчивые коды, такие как БЧХ, Хемминга, Рида-Соломона и Голея, а также актуальность использования этих кодов.

Разделы основной части:

- 1 В разделе «Анализ алгоритмов помехоустойчивых каналов связи» проводится подробный анализ алгоритмов помехоустойчивых кодов, включая основные принципы их работы, отличительные особенности, этапы кодирования.
- 2 В разделе «Программная реализация модели» дается обоснование выбора среды разработки, архитектуры приложения, описываются основные методы и классы программы, а также показывается, как выглядит программное средство.
- 3 В разделе «Работоспособность программного средства» представлены примеры работы программы, демонстрируется работоспособность, входные и выходные данные, а также рассматриваются преимущества и недостатки программного средства.

Заключение:

Результатом работы станет программный инструмент, сочетающий в себе образовательную ценность и возможность практической проверки теоретических знаний.

Руководитель работы

подпись, дата

Р. В. Егорова

Задание принял к исполнению

подпись, дата

И. А. Ковалец

Содержание

Введение.....	5
1 Анализ алгоритмов помехоустойчивого кодирования.....	7
1.1 Алгоритм кодов Хэмминга.....	7
1.2 Алгоритм кодов Рида-Соломона	9
1.3 Алгоритм кодов Голея	10
1.4 Алгоритм кодов БЧХ	12
2 Программная реализация модели	14
2.1 Выбор языка программирования, инструментов и архитектуры.....	14
2.2 Основные этапы работы программы.....	15
2.3 Реализация алгоритма кодирования.....	17
2.4 Реализация алгоритма декодирования	20
2.5 Особенности и ограничения реализации	23
3 Работоспособность программного средства	24
Заключение	28
Перечень использованных информационных ресурсов	29
Приложение А Листинг кода.....	30

					ТКСВИ.140000.000 ПЗ							
Изм.	Лист	№ докум.	Подпись	Дата	Модель помехоустойчивого канала связи на основе БЧХ-кодов Пояснительная записка			Лит.	Лист	Листов		
Разраб.		Ковалец И.А.										
Провер.		Егорова Р.В.								4	33	
								ДГТУ				
Н.контр.								Кафедра КБИС				
Утв.		Сафарьян О.А.										

Введение

Современные системы передачи и хранения информации работают в условиях наличия помех, которые могут искажать передаваемые данные. Ошибки, возникающие в каналах связи из-за шумов, помех, сбоев оборудования или преднамеренного воздействия, способны существенно снижать достоверность информации. Для обеспечения надёжности передачи данных в цифровых системах широко применяются методы помехоустойчивого кодирования, позволяющие не только обнаруживать, но и исправлять ошибки без необходимости повторной передачи. Многие из них до сих пор используются в реальных системах: от оперативной памяти с ECC и QR-кодов до космической связи и цифрового телевидения.

Объектом данной курсовой работы являются помехоустойчивые коды Хэмминга, коды БЧХ, Рида–Соломона и Голея.

Предметом данной работы является разработка программного средства – демонстрационная модель помехоустойчивого канала связи.

Целью курсовой работы является разработка программного средства, демонстрирующего работу классического помехоустойчивого кода, с возможностью пошагового наблюдения за процессами кодирования и исправления ошибок, а также проведение сравнительного анализа эффективности рассматриваемых кодов.

Для достижения поставленной цели решаются следующие задачи:

1 провести теоретический анализ принципов построения и работы помехоустойчивых кодов;

2 разработать программное приложение реализующее кодирование и декодирование помехоустойчивого кода с выводом всех промежуточных этапов;

3 проверить разработанное программное средство на работоспособность;

В первой главе работы проводится подробный анализ алгоритмов помехоустойчивого кодирования Хэмминга, циклических кодов БЧХ, кодов Рида–Соломона и кода Голея: рассмотреть их математическую основу, этапы кодирования

					ТКСВИ.140000.000 ПЗ	Лист
						5
Изм.	Лист	№ докум.	Подпись			

и декодирования: рассматриваются математические основы, принципы построения порождающих и проверочных матриц, особенности синдромного декодирования для каждого из кодов. Далее описывается программная реализация: обосновывается выбор языка программирования и среды разработки, представлена архитектура приложения, описываются основные классы и методы, а также демонстрируется внешний вид разработанного программного средства.

					ТКСВИ.140000.000 ПЗ	Лист
						6
Изм.	Лист	№ докум.	Подпись			

1 Анализ алгоритмов помехоустойчивого кодирования

С развитием цифровых технологий объёмы передаваемой и хранимой информации быстро растут, а каналы связи подвержены помехам: шумам, искажениям и сбоям. Это приводит к ошибкам в данных, снижая их достоверность. Ключевым решением является помехоустойчивое кодирование – введение избыточности для обнаружения и исправления ошибок без повторной передачи.

Помехоустойчивое кодирование использует линейные блочные коды, добавляя проверочные символы для контроля целостности. Такие коды позволяют полностью восстановить сообщение при ограниченном числе ошибок [1].

Классическими примерами являются коды Хэмминга, рассчитанные для одиночных ошибок, Рида – Соломона – для пакетных ошибок, совершенный код Голея и БЧХ являющимися гибкими циклическими кодами с множественным исправлением.

1.1 Алгоритм кодов Хэмминга

Коды Хэмминга были разработаны американским математиком Ричардом Уэсли Хэммингом в конце 1940-х годов во время его работы в Bell Labs. В 1950 году он опубликовал фундаментальную статью "Error Detecting and Error Correcting Codes", где описал принцип построения кодов для автоматического исправления ошибок. Этот прорыв был вызван практическими проблемами: в те времена релейные компьютеры часто давали сбои из-за шумов в механических контактах, и Хэммингу пришлось вручную искать ошибки в перфокартах. Его код стал одним из первых практических помехоустойчивых кодов и заложил основу для всей современной теории кодирования.

Коды Хэмминга представляют собой линейные блочные двоичные коды с минимальным расстоянием три, предназначенные для исправления одиночных ошибок и обнаружения двойных. Они являются совершенными кодами, то есть оптимально используют избыточность для заданной исправляющей способности [2].

Типичные параметры – длина кодового слова, равная разнице двойке в степени r и единицы, где r – число проверочных битов, а число информационных битов равно разнице длине кодового слова и r .

Алгоритм кодирования работает следующим образом. Биты данных размещаются в позициях кодового слова начиная с 1, которые не являются степенями двойки. Проверочные биты вставляются в позиции-степени двойки и вычисляются так, чтобы обеспечивать чётность в определённых группах битов: каждый проверочный бит отвечает за позиции, где его номер участвует в двоичном представлении номера бита. Таким образом, добавляется минимально необходимая избыточность для контроля всех позиций.

Декодирование начинается с повторного вычисления проверочных битов на принятом кодовом слове. Разница между вычисленными и принятыми проверочными битами формирует синдром – число, которое в двоичном виде указывает на позицию ошибочного бита. Если синдром нулевой – ошибок нет, но, если синдром ненулевой, то бит в указанной позиции инвертируется, исправляя одиночную ошибку. Расширенный вариант добавляет общий бит чётности для обнаружения двойных ошибок.

Для оценки эффективности кодов Хэмминга используются такие показатели, как отношение числа информационных битов к общей длине кода (кодовая скорость), вероятность неисправленной ошибки при заданном уровне шума, а также скорость обработки данных.

Достоинства:

1. Простота и высокая скорость реализации.
2. Минимальная избыточность для исправления одной ошибки.
3. Широкое применение в реальных системах.

Недостатки:

1. Исправляет только одиночные ошибки.
2. Неэффективен при множественных или пакетных ошибках.

Коды Хэмминга остаются фундаментальным примером помехоустойчивого кодирования, демонстрируя оптимальный баланс между простотой и эффективностью для задач с одиночными ошибками.

1.2 Алгоритм кодов Рида-Соломона

Коды Рида–Соломона были разработаны в 1960 году американскими учёными Ирвингом Ридом и Гюставом Соломоном в лаборатории МИТ. Алгоритм изначально создавался для исправления ошибок в системах передачи данных, включая космическую связь, и быстро стал одним из самых мощных инструментов помехоустойчивого кодирования.

Коды Рида–Соломона представляют собой небинарные циклические блочные коды, работающие над конечным полем Галуа $GF(2^m)$, где каждый символ – это группа из m битов, обычно m берется 8. Они позволяют исправлять множественные ошибки и стирания в блоках данных фиксированной длины, с высокой эффективностью при пакетных ошибках [3].

Алгоритм кодирования работает следующим образом. Исходное сообщение рассматривается как полином высокой степени над полем Галуа. Для добавления избыточности сообщение делится на специально построенный порождающий полином, корни которого являются последовательными степенями примитивного элемента поля. Остаток от деления добавляется к сообщению в виде проверочных символов. Таким образом, кодовое слово становится кратным порождающему полиному, что обеспечивает контроль ошибок.

Декодирование начинается с вычисления синдромов – значений принятого полинома в корнях порождающего полинома. Если синдромы нулевые, ошибок нет. При наличии ошибок решается система уравнений для поиска полинома локаторов ошибок. Затем определяется позиция и величина каждой ошибки поиском корней этого полинома, после чего ошибки исправляются вычитанием из принятых символов.

Для оценки эффективности кодов Рида–Соломона используются такие

					ТКСВИ.140000.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись			9

показатели, как отношение числа информационных символов к общей длине блока, вероятность неисправленной ошибки при заданном уровне шума, способность исправлять стирания и скорость обработки.

Достоинства:

1. Высокая исправляющая способность при пакетных ошибках и стираниях.
2. Точный контроль над числом исправляемых ошибок.
3. Широкое применение в реальных системах.

Недостатки:

1. Высокая вычислительная сложность из-за операций в конечном поле.
2. Чувствительность к превышению исправляющей способности.

Коды Рида–Соломона остаются одним из наиболее мощных и универсальных средств помехоустойчивого кодирования, особенно эффективных в задачах с пакетными ошибками и стираниями данных.

1.3 Алгоритм кодов Голея

Коды Голея были открыты швейцарским математиком Марселем Голеем в 1949 году во время его работы в области теории информации. Он описал два уникальных совершенных кода: бинарный код длиной 23 и троичный код длиной 11. Бинарный код Голея (23,12,7) был опубликован и стал одним из первых примеров совершенных кодов, достигающих границы Хэмминга. Этот код сразу привлёк внимание теоретиков, так как доказывал существование оптимальных кодов для исправления трёх ошибок, и позже нашёл применение в космических миссиях.

Коды Голея представляют собой линейные блочные совершенные коды с высокой исправляющей способностью. Бинарный код Голея имеет параметры (23,12,7): длина кодового слова 23 бита, 12 информационных битов, минимальное расстояние 7, что позволяет исправлять до 3 ошибок [4]. Существует также расширенный вариант (24,12,8) с расстоянием 8. Эти коды уникальны тем, что являются единственными нетривиальными совершенными двоичными кодами известной длины.

					ТКСВИ.140000.000 ПЗ	Лист
						10
Изм.	Лист	№ докум.	Подпись			

Алгоритм кодирования работает следующим образом. Код Голя можно представить как циклический код с фиксированным порождающим полиномом степени 11, который делит все кодовые слова. Информационные биты умножаются на порождающую матрицу или делится полиномиальное представление сообщения, и к результату добавляются проверочные биты в виде остатка от деления. Альтернативно используется систематическая форма: информационные биты размещаются в фиксированных позициях, а проверочные вычисляются через линейные комбинации сложение по модулю 2 по заранее определённым группам, обеспечивая вес кодовых слов кратным определённым значениям.

Декодирование использует синдромный подход с перебором возможных ошибок, поскольку при t равное 3 количество вариантов около 2 тысяч, то используются быстрые алгебраические алгоритмы, которые выполняют несколько сотен или тысяч операций на одно кодовое слово. Сначала вычисляется синдром по проверочной матрице. Если синдром нулевой – ошибок нет. Далее перебираются варианты исправления 0–3 битов, пока не найдётся кодовое слово с нулевым синдромом или используется таблица синдромов для быстрого поиска. Более продвинутые методы опираются на алгебраическую структуру кода позволяя декодировать без полного перебора.

Для оценки эффективности кодов Голя используются такие показатели, как кодовая скорость – это отношение информационных битов к общей длине, вероятность неисправленной ошибки в канале с шумом, а также вычислительная сложность декодирования по сравнению с другими кодами аналогичной исправляющей способности.

Достоинства:

1. Оптимальность как совершенного кода (максимальная эффективность для заданных параметров).
2. Высокая исправляющая способность ($t=3$) при умеренной избыточности.
3. Хорошая устойчивость к случайным ошибкам.

Недостатки:

1. Фиксированные параметры (только 23/24 бита), что ограничивает гибкость.
2. Более высокая сложность декодирования по сравнению с простыми кодами вроде Хэмминга.

Коды Голея представляют собой яркий пример теоретически оптимального помехоустойчивого кодирования и остаются важным объектом изучения в теории кодов, несмотря на ограниченное практическое применение из-за фиксированной длины.

1.4 Алгоритм кодов БЧХ

Коды БЧХ (Боуза–Чоудхури–Хоквингема) были независимо предложены в 1959–1960 годах тремя учёными: Алексисом Хоквингемом, а также Раджойем Боузом и Диллипом Чоудхури. Их работы объединили идеи циклических кодов с алгебраической теорией конечных полей, позволив создавать коды с произвольной заданной исправляющей способностью. Коды БЧХ быстро стали важным обобщением более ранних циклических кодов и нашли применение в системах связи, где требуется исправление множественных ошибок.

Коды БЧХ представляют собой мощный класс двоичных циклических блочных кодов над конечным полем, позволяющих исправлять заданное число случайных ошибок t при длине блока $n = 2^m - 1$. Они обладают гибкими параметрами: число проверочных битов примерно равно $m \cdot t$, а минимальное расстояние гарантированно не менее $2t + 1$.

Алгоритм кодирования работает следующим образом. Исходное сообщение представляется как полином над $GF(2)$. Порождающий полином $g(x)$ строится как наименьшее общее кратное минимальных полиномов для последовательных корней – степеней примитивного элемента поля. Кодовое слово формируется путём умножения информационного полинома на $g(x)$ или, в систематической форме, добавлением остатка от деления сообщения на $g(x)$ в конец блока. Таким образом,

каждое кодовое слово становится кратным порождающему полиному.

Декодирование начинается с вычисления синдромов — значений принятого полинома в заданных корнях порождающего полинома что обычно составляет $2t$ синдромов. Если все синдромы нулевые, ошибок нет. При наличии ошибок решается ключевое уравнение для поиска полинома локаторов ошибок. Затем корни этого полинома определяют позиции ошибок, а величины ошибок вычисляются дополнительно. Для малых t возможны упрощённые методы с циклическими сдвигами и прямым перебором синдромов.

Для оценки эффективности кодов БЧХ используются такие показатели, как кодовая скорость — это отношение информационных битов к общей длине, вероятность неисправленной ошибки в канале с шумом, а также сложность вычислений в зависимости от исправляющей способности t .

Достоинства:

1. Гибкость в выборе исправляющей способности t при фиксированной длине блока.
2. Хорошая эффективность для случайных множественных ошибок.
3. Возможность упрощённых реализаций для малого t .

Недостатки:

1. Сложность декодирования быстро растёт с увеличением t из-за операций в расширенном поле.
2. Требуется точное знание параметров поля для построения порождающего полинома.

Коды БЧХ являются одним из наиболее универсальных и мощных классов циклических кодов, позволяющих эффективно решать задачи помехоустойчивого кодирования с множественным исправлением ошибок.

2 Программная реализация модели

2.1 Выбор языка программирования, инструментов и архитектуры

Для реализации программного средства был выбран язык программирования C# в сочетании со средой разработки Microsoft Visual Studio и платформой Windows Forms для создания графического интерфейса. Выбор C# обусловлен его высокой производительностью, строгой типизацией и богатым набором стандартных библиотек, что обеспечивает надёжность и удобство разработки приложений под Windows. Язык обладает отличной поддержкой объектно-ориентированного программирования, что позволяет создавать чистый, модульный и легко поддерживаемый код. Кроме того, C# интегрируется с .NET Framework, предоставляя мощные средства работы со строками, массивами и битовыми операциями, которые критически важны при реализации алгоритмов помехоустойчивого кодирования. Для данной задачи, связанной с обработкой двоичных данных, вычислением матриц, синдромов и пошаговой визуализацией процессов кодирования и декодирования, C# оказался оптимальным благодаря простоте работы с битовыми операциями такими как XOR и сдвиги, высокой скорости выполнения и наличию удобных элементов управления в Windows Forms – RichTextBox, DataGridView и другие, позволяющие наглядно отображать промежуточные результаты. Visual Studio обеспечивает отличную отладку, интуитивный дизайнер форм и быструю компиляцию, что существенно ускорило процесс разработки демонстрационной модели.

Архитектура приложения построена по классической схеме Windows Forms с одной основной формой, содержащей все элементы управления. Вся логика кодирования и декодирования сосредоточена в классе формы, что упрощает доступ к элементам интерфейса и позволяет оперативно обновлять текстовые поля и таблицы в процессе выполнения алгоритма. Такой подход выбран сознательно для демонстрационных целей: он обеспечивает максимальную прозрачность работы программы, позволяя пользователю наблюдать каждый шаг вычислений в реальном времени без лишней абстракции.

					ТКСВИ.140000.000 ПЗ	Лист
						14
Изм.	Лист	№ докум.	Подпись			

Особенностью реализации стало активное использование элементов DataGridView для визуализации порождающей матрицы G и пошагового процесса декодирования. Матрица строится динамически в зависимости от введенных параметров n и порождающего полинома $g(x)$, а её отображение помогает пользователю понять структуру циклического кода.

Кроме того, применён параллельный подход к кодированию блоков данных с помощью Parallel.For, что ускоряет обработку длинных сообщений и демонстрирует возможности современного C# в многопоточности.

Ещё одной важной особенностью является подробный пошаговый вывод процесса декодирования в отдельном RichTextBox. Программа фиксирует каждое вычисление синдрома, циклический сдвиг кодового слова, исправление битов и извлечение информационного слова, что делает её ценным образовательным инструментом. Такой детальный лог позволяет не только проверить корректность работы, но и изучить механизм синдромного декодирования БЧХ-кодов на конкретных примерах, включая случаи с одиночными и множественными ошибками в пределах исправляющей способности.

2.2 Основные этапы работы программы

Разработанное программное средство представляет собой демонстрационную модель помехоустойчивого канала связи на основе циклических БЧХ-кодов с графическим интерфейсом на базе Windows Forms. Работа программы построена на последовательном выполнении этапов, обеспечивающих пошаговую визуализацию процессов кодирования и декодирования. Для пояснения функционирования приложения можно выделить следующие основные этапы:

1. Загрузка и обработка входных данных. Пользователь открывает текстовый файл (в кодировке UTF-8) с помощью кнопки «Открыть файл». Программа читает содержимое файла, отображает исходный текст в соответствующем поле и автоматически преобразует его в двоичную последовательность по 8 бит на символ. Эта последовательность

выводится в поле информационного сообщения и служит основой для дальнейшего кодирования.

2. Ввод параметров кода и расчёт характеристик. Пользователь вводит порождающий полином $g(x)$ в двоичном виде и длину кодового слова n . Программа автоматически вычисляет степень полинома m , длину информационного слова k , а также исправляющую способность t на основе встроенной таблицы известных параметров БЧХ-кодов. Динамически строится и отображается порождающая матрица G в элементе DataGridView, что позволяет визуальнo контролировать структуру кода.
3. Кодирование сообщения. После нажатия кнопки «Кодировать» программа добавляет необходимые нулевые биты для кратности k , разбивает двоичную последовательность на блоки длиной k и выполняет кодирование каждого блока путём умножения на матрицу G с использованием операций XOR. Процесс оптимизирован с применением параллельных вычислений (Parallel.For). Закодированная последовательность отображается в соответствующем поле, а также сохраняется для последующих этапов.
4. Искусственное внесение ошибок. Пользователь может вручную или автоматически внести ошибки в закодированную последовательность, инвертируя отдельные биты. Это имитирует помехи в канале связи и позволяет демонстрировать работу механизма исправления ошибок в контролируемых условиях.
5. Декодирование с исправлением ошибок. При нажатии кнопки «Декодировать» программа обрабатывает каждый кодовый блок: вычисляет синдром путём деления на $g(x)$, выполняет циклические сдвиги до достижения синдрома с весом не более t , исправляет ошибки вычитанием синдрома и извлекает информационное слово. Весь процесс подробно протоколируется в отдельном поле с указанием каждого шага.

В итоге восстанавливается исходное сообщение и преобразуется обратно в текст.

Таким образом, программа обеспечивает полный цикл моделирования помехоустойчивого канала связи: от ввода данных до восстановления сообщения с исправлением ошибок. Пошаговая визуализация и детальный лог делают её эффективным образовательным инструментом для изучения принципов работы БЧХ-кодов.

2.3 Реализация алгоритма кодирования

Алгоритм кодирования реализует несистематическую форму циклического кода на основе заданного порождающего полинома. В этой форме информационные биты не выделяются явно в кодовом слове в неизменённом виде — они равномерно распределены по всему кодовому слову за счёт циклических сдвигов и линейных комбинаций. Такой подход напрямую отражает алгебраическую структуру циклических кодов и удобен для демонстрации их свойств. Процесс выполняется поблочно и включает несколько последовательных этапов, направленных на подготовку данных, построение структуры кода и формирование кодовых слов. Ниже приведено детальное описание каждого этапа.

1. Подготовка исходного сообщения:

- Исходное текстовое сообщение преобразуется в сплошную двоичную последовательность, где каждый символ представляется ровно восьмью битами.
- Вычисляется точная длина этой последовательности в битах.
- Чтобы обеспечить равномерное разбиение на блоки, определяется количество нулевых битов, необходимых для дополнения до длины, кратной длине информационного слова (это значение рассчитывается как остаток от деления с учётом модуля).

- Нулевые биты добавляются строго в конец последовательности, формируя дополненное сообщение, которое отображается пользователю для контроля.

2. Определение параметров кода и построение порождающей матрицы:

- На основе введенного порождающего полинома определяется его степень — позиция старшего ненулевого коэффициента.
- Длина информационного слова вычисляется как разность между общей длиной кодового слова и степенью полинома.
- Строится порождающая матрица размером (длина информационного слова) $\times$ (длина кодового слова): каждая строка матрицы представляет собой циклический сдвиг порождающего полинома вправо (или влево) внутри окна длины кодового слова, с заполнением нулями в остальных позициях.
- Полученная матрица является несистематической, но полностью эквивалентной систематической форме по свойствам кода.

3. Разбиение сообщения на информационные блоки:

- Дополненная двоичная последовательность делится на отдельные блоки фиксированной длины, равной длине информационного слова.
- Вычисляется общее количество таких блоков, чтобы охватить всё сообщение без остатка.

4. Кодирование каждого информационного блока:

- Для каждого блока создается пустое кодовое слово длиной равной полной длине кода, первоначально заполненное нулями.
- Последовательно просматриваются все биты информационного блока: если текущий бит равен 1, то к текущему кодовому слову добавляется соответствующая строка порождающей матрицы. Если бит равен 0, строка матрицы не добавляется.

- В результате накопления таких операций формируется полное кодовое слово: оно представляет собой линейную комбинацию сдвигов порождающего полинома, соответствующих позициям единиц в информационном блоке.
- Этот процесс эквивалентен умножению вектора информационных битов на порождающую матрицу в поле $GF(2)$.

5. Параллельная обработка блоков для ускорения:

- Кодирование отдельных блоков выполняется одновременно на нескольких вычислительных ядрах, поскольку операции над блоками независимы.
- Это позволяет значительно сократить время обработки для длинных сообщений, сохраняя последовательность результатов.

6. Сборка и вывод полного закодированного сообщения:

- После завершения кодирования всех блоков полученные кодовые слова соединяются в одну непрерывную двоичную последовательность в порядке исходных блоков.
- Итоговое закодированное сообщение отображается в интерфейсе программы как единая строка битов.
- Дополнительно обновляются визуальные элементы: параметры кода, матрица и промежуточные данные для полного контроля процесса.

Данный подход обеспечивает точное соответствие теоретическим принципам циклических кодов в неканонической форме, где кодовые слова являются кратными порождающему полиному. Несистематическая форма выбрана намеренно для наглядной демонстрации циклической структуры, хотя эквивалентная систематическая форма также возможна путём приведения матрицы. Алгоритм прозрачен, эффективен и идеально подходит для образовательных целей, позволяя пользователю наблюдать преобразование сообщения в защищённую форму.

2.4 Реализация алгоритма декодирования

Алгоритм декодирования реализует декодер Меггита в упрощённой форме с перебором циклических сдвигов, подходящий для демонстрации работы циклических БЧХ-кодов с малой исправляющей способностью. Декодер Меггита, предложенный Джоном Меггитом в 1960-х годах, использует свойство циклической структуры кода для последовательного сдвига принятого слова и пересчёта синдрома до тех пор, пока синдром не соответствует шаблону исправимой ошибки. В реализованной версии этот подход адаптирован для учебных целей: вместо полной таблицы шаблонов ошибок используется условие на вес синдрома, что делает метод простым и наглядным. Декодирование выполняется поблочно для всех принятых сообщений в том числе и без ошибочных и включает детальную визуализацию каждого шага. Ниже приведено детальное описание каждого этапа.

1. Подготовка принятого сообщения:

- Принятая двоичная последовательность, возможно с внесёнными ошибками, берётся из поля с закодированным сообщением.
- Вычисляется общая длина последовательности и определяется количество кодовых блоков как частное от деления длины на длину кодового слова.
- Последовательность разбивается на отдельные кодовые слова фиксированной длины, соответствующие параметру кода.

2. Обработка каждого кодового блока по отдельности:

- Для каждого блока создаётся рабочая копия кодового слова.
- Инициализируется счётчик итераций и переменная для хранения текущего синдрома.
- Весь процесс для одного блока протоколируется в отдельном поле вывода с указанием номера блока и промежуточных данных.

3. Вычисление начального синдрома:

					ТКСВИ.140000.000 ПЗ	Лист
						20
Изм.	Лист	№ докум.	Подпись			

- Синдром вычисляется как остаток от деления текущего кодового слова на порождающий полином в поле $GF(2)$, через последовательные операции XOR.
- Если синдром состоит из всех нулей, ошибок в блоке нет — переход к извлечению информационного слова.
- Вычисляется вес синдрома, т.е. количество единиц в нём.

4. Циклический поиск правильного положения ошибки:

- Пока вес синдрома превышает заданную исправляющую способность, выполняется циклический сдвиг кодового слова влево.
- После каждого сдвига заново вычисляется синдром для сдвинутого слова.
- Сдвиг продолжается до тех пор, пока вес синдрома не станет равным или меньшим исправляющей способности — это соответствует ошибке в стандартном положении.
- Каждый сдвиг и соответствующий синдром фиксируются в протоколе для наглядности, демонстрируя последовательный перебор возможных положений ошибки.

5. Исправление ошибок:

- Когда найден подходящий синдром, он используется как маска ошибки в захваченном положении.
- Синдром складывается по модулю 2 на соответствующие позиции кодового слова с учётом выполненных сдвигов.
- Для каждой единицы в синдроме инвертируется бит в кодовом слове по рассчитанной позиции.
- После исправления кодовое слово циклически сдвигается обратно на количество выполненных сдвигов, чтобы восстановить исходный порядок.

- Если синдром был нулевым изначально, этот этап пропускается с сообщением об отсутствии ошибок.

6. Извлечение информационного слова:

- Из исправленного кодового слова удаляются проверочные биты путём деления на порождающий полином (остаток отбрасывается, частное — информационное слово).
- Полученное информационное слово длиной равной длине информационного блока добавляется к общему восстановленному сообщению.
- Информационное слово отображается в протоколе для текущего блока.

7. Сборка полного восстановленного сообщения:

- После обработки всех блоков собранная последовательность информационных слов очищается от добавленных при кодировании нулевых битов.
- Восстановленная двоичная последовательность преобразуется обратно в текст: биты группируются по восемь, преобразуются в байты и декодируются как текст в соответствующей кодировке.
- Итоговый текст отображается пользователю как расшифрованное сообщение.

8. Визуализация и протоколирование:

- Весь процесс декодирования подробно логируется: для каждого блока выводятся принятое кодовое слово, все промежуточные синдромы, количество сдвигов, исправленные биты, исправленное кодовое слово и извлечённое информационное слово.
- Это обеспечивает полную прозрачность и позволяет пользователю шаг за шагом проследить за выполнением.

Данный подход представляет собой классический декодер Меггита в упрощённой учебной реализации, эффективный для кодов с малой исправляющей

					<i>ТКСВИ.140000.000 ПЗ</i>	Лист
						22
Изм.	Лист	№ докум.	Подпись			

способностью, где перебор всех возможных сдвигов остаётся вычислительно приемлемым. Он точно отражает принцип поиска маски ошибки и идеально подходит для демонстрации: пользователь видит, как программа последовательно сдвигает слово, ищет ошибку и исправляет её в пределах заданной способности.

2.5 Особенности и ограничения реализации

Разработанное программное средство обладает рядом особенностей, повышающих его образовательную ценность: подробная пошаговая визуализация процессов кодирования и декодирования, параллельная обработка блоков для ускорения работы с длинными сообщениями, а также возможность искусственного внесения ошибок для демонстрации исправления. Это делает приложение удобным инструментом для изучения принципов помехоустойчивого кодирования.

Вместе с тем, реализация имеет ограничения: упрощённый декодер Меггита эффективен только для кодов с малой исправляющей способностью, так как перебор сдвигов становится вычислительно затратным при больших n и t , используется несистематическая форма кода, отсутствует поддержка очень больших файлов без дополнительной оптимизации памяти, а расчёт исправляющей способности ограничен фиксированной таблицей известных параметров БЧХ-кодов. Эти ограничения обусловлены демонстрационным характером программы и могут быть устранены в дальнейших доработках.

3 Работоспособность программного средства

Для запуска разработанного программного средства требуется операционная система Microsoft Windows с установленной платформой .NET Framework версии 4.8 или выше. Приложение представляет собой исполняемый файл, созданный в среде Visual Studio, и не требует дополнительной установки — достаточно запустить файл exe. После запуска открывается главное окно с графическим интерфейсом на базе Windows Forms.

Работа с программой начинается с загрузки текстового файла: пользователь нажимает кнопку «Открыть файл», выбирает файл в кодировке UTF-8, после чего исходный текст отображается в соответствующем поле, а его двоичное представление — в поле информационного сообщения, что изображено на рисунке 1.

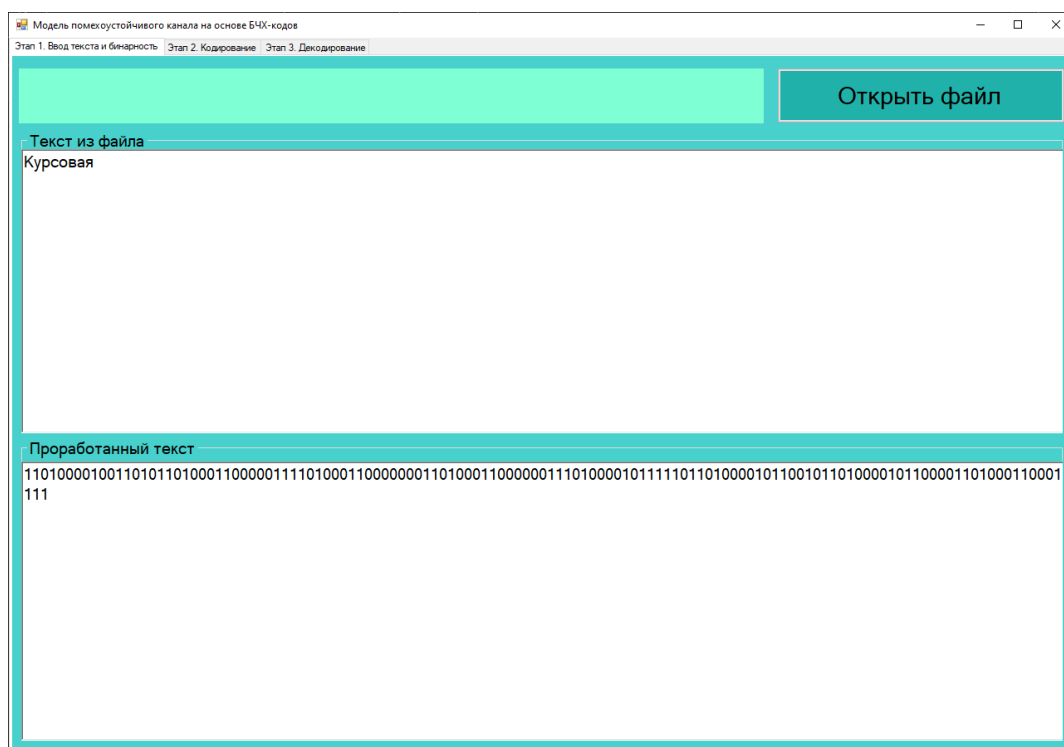


Рисунок 1 – Вставленные и обработанные слова

Далее необходимо ввести параметры кода: порождающий полином $g(x)$ в двоичном виде, например – 1011 и длину кодового слова $n = 7$. Программа автоматически рассчитывает степень полинома m , длину информационного слова k ,

исправляющую способность s и строит порождающую матрицу G , отображаемую в таблице. После нажатия кнопки «Кодировать» выполняется кодирование сообщения, результат отображается в поле закодированного текста, что показано на рисунке 2.

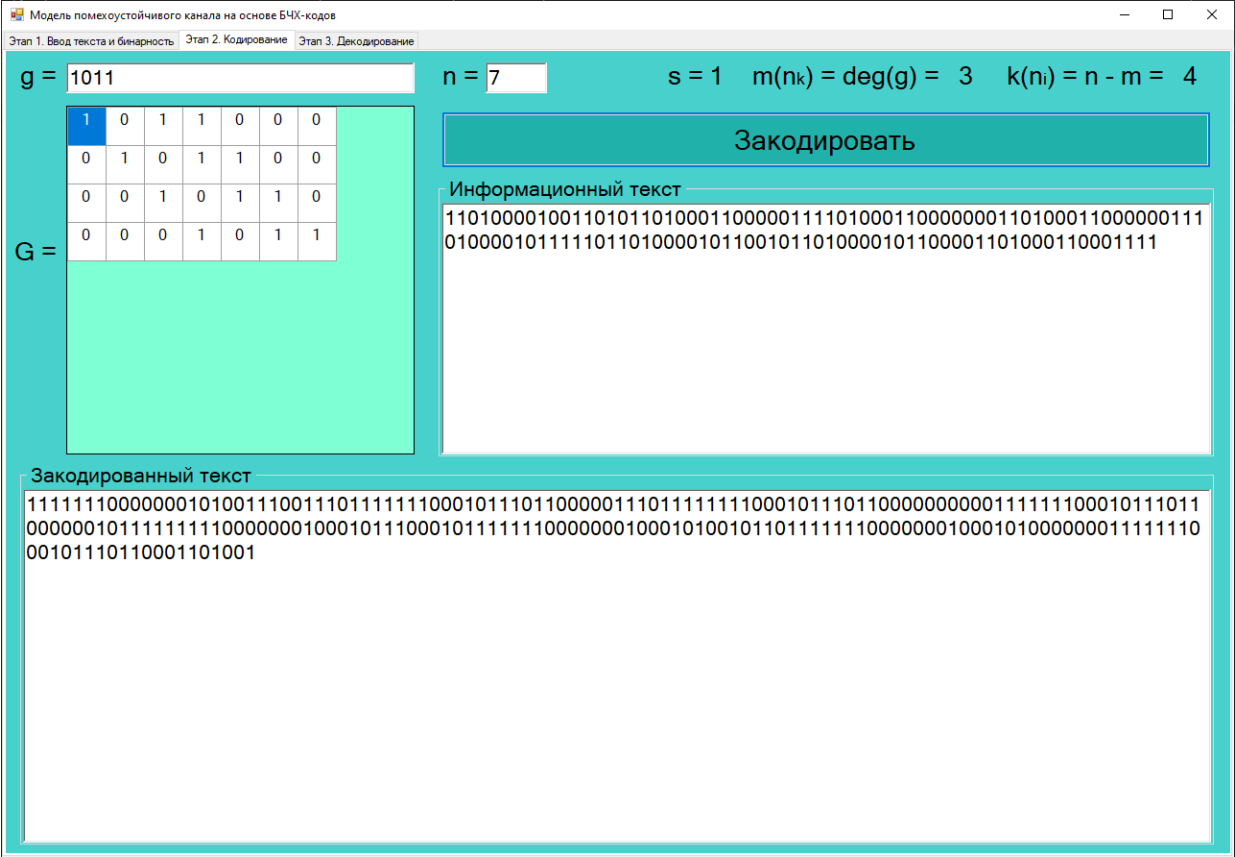
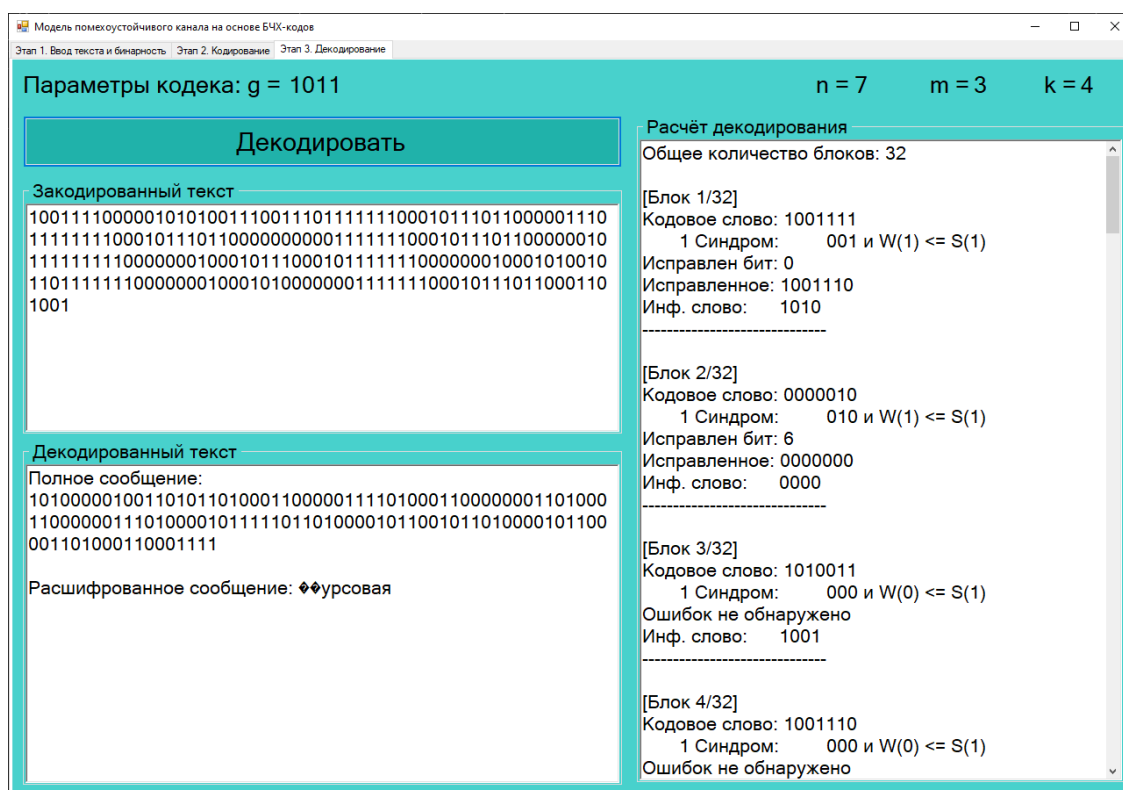
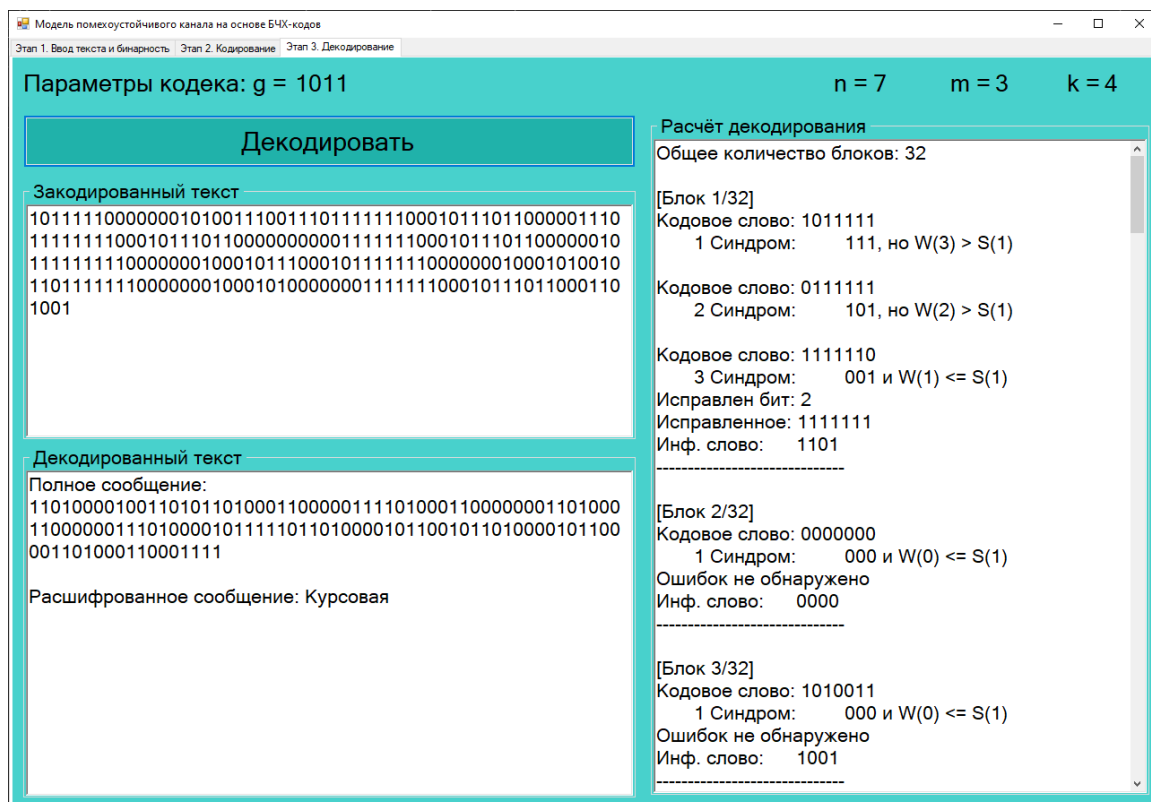


Рисунок 2 – Окно программы после ввода параметров кода

Пользователь может вручную внести ошибки в закодированную последовательность для имитации помех в канале связи. Затем нажатием кнопки «Декодировать» запускается процесс декодирования по алгоритму Меггита. Программа выводит подробный лог для каждого блока, включая синдромы, сдвиги, исправленные биты и извлечённые информационные слова. В итоге отображается восстановленный текст, что изображено на рисунке 3.



Анализ работы программы показывает её полную работоспособность в рамках поставленных задач. При корректных параметрах кодирование выполняется быстро даже для текстов объёмом несколько тысяч символов благодаря параллельной обработке. Декодер Меггита успешно исправляет ошибки в пределах заданной способности t , демонстрируя пошагово механизм захвата и исправления. В случаях превышения t или некорректных параметров программа корректно обрабатывает ситуацию что видно на рисунке 4, что позволяет иллюстрировать границы метода.

					<i>ТКСВИ.140000.000 ПЗ</i>	Лист
						27
Изм.	Лист	№ докум.	Подпись			

Заключение

В рамках данной курсовой работы была разработана демонстрационная модель помехоустойчивого канала связи на основе циклических БЧХ-кодов в виде desktop-приложения на языке C# с графическим интерфейсом Windows Forms. Программное средство позволяет загружать текстовые данные, задавать параметры кода, выполнять кодирование в несистематической форме с использованием порождающей матрицы, имитировать помехи в канале и декодировать сообщение с исправлением ошибок по упрощённому алгоритму Меггита. Реализация обеспечивает полную пошаговую визуализацию всех этапов, что делает приложение ценным образовательным инструментом для изучения принципов помехоустойчивого кодирования.

В ходе выполнения работы были решены следующие задачи:

1. Проведён теоретический анализ классических помехоустойчивых кодов (Хэмминга, Рида–Соломона, Голея и БЧХ), включая принципы их построения, кодирования и декодирования, с акцентом на сравнительные характеристики.
2. Разработано программное приложение на C#, реализующее кодирование и декодирование БЧХ-кодов с визуализацией порождающей матрицы, синдромов и процесса исправления ошибок.
3. Обеспечена возможность искусственного внесения ошибок и демонстрации работы декодера в пределах исправляющей способности.
4. Проведено тестирование работоспособности программы на различных примерах, подтвердившее корректность реализации и наглядность представления результатов.

Итогом курсовой работы стало создание работоспособного и информативного программного средства, сочетающего теоретическую основу с практической демонстрацией. Приложение успешно моделирует помехоустойчивый канал связи, позволяет пользователю наглядно наблюдать механизмы обнаружения и исправления ошибок в циклических БЧХ-кодах и может быть использовано в образовательных целях.

					ТКСВИ.140000.000 ПЗ	Лист
						28
Изм.	Лист	№ докум.	Подпись			

Перечень использованных информационных ресурсов

1. Березкин Е. Ф. Основы теории информации и кодирования. Учебное пособие для вузов. – Москва : ЛАНЬ, 2021. – 321 с.
2. Трифонов П.В. Основы помехоустойчивого кодирования: учебное пособие. - Санкт-Петербург: Университет ИТМО, 2022. - 231 с.
3. Вернер М. Основы кодирования. Учебник для ВУЗов. - Москва: Техносфера, 2004. - 288с.
4. Морелос-Сарагоса Р. Искусство помехоустойчивого кодирования. Методы алгоритмы, применение. - Москва: Техносфера, 2005. - 320с.

					ТКСВИ.140000.000 ПЗ	Лист
						29
Изм.	Лист	№ докум.	Подпись			

Приложение А Листинг кода

```
private void createMatrix(int row, int col, DataGridView Matrix) //
Функция для создания DataGridView по размеру

{
    DataGridViewColumn column;
    for (int i = 0; i < col; i++)
    {
        column = new DataGridViewTextBoxColumn();
        column.Width = 40;
        Matrix.Columns.Add(column);
    }
    for (int i = 0; i < row; i++)
    {
        Matrix.Rows.Add();
        Matrix.Rows[i].Height = 40;
    }
}

private int[,] BuildGeneratorMatrix(string g, int n, int k, int m) //
Функция создания матрицы G

{
    // 1. Создаем двумерный массив для хранения матрицы
    int[,] matrix = new int[k, n];

    // 2. Преобразуем строку g в список битов
    int[] gBits = new int[g.Length];
    for (int i = 0; i < g.Length; i++)
    {
        gBits[i] = g[i] - '0';
    }
}
```

					ТКСВИ.140000.000 ПЗ	Лист
						30
Изм.	Лист	№ докум.	Подпись			

```

// 3. Заполняем матрицу
for (int row = 0; row < k; row++)
{
    for (int col = 0; col < n; col++)
    {
        matrix[row, col] = 0;
        DGW_Matrix_1.Rows[row].Cells[col].Value = "0";
    }
    for (int i = 0; i < gBits.Length; i++)
    {
        int pos = row + i;
        if (pos < n)
            matrix[row, pos] = gBits[i];
        DGW_Matrix_1.Rows[row].Cells[pos].Value = gBits[i];
    }
}

return matrix;
}

private string MultiplyInfoWordWithMatrix(int[,] matrix, int[]
infoWord) // Функция перемножения матрицы G и инф. слова
{
    int rows = matrix.GetLength(0);      // количество строк
    int cols = matrix.GetLength(1);      // количество столбцов
    int[] result = new int[cols];
    // Инициализируем нулями
    for (int j = 0; j < cols; j++)
        result[j] = 0;

    // Перемножаем по правилу циклических кодов (XOR для строк с
infoWord[i] == 1)
    for (int i = 0; i < rows && i < infoWord.Length; i++)

```

```

{
    if (infoWord[i] == 1)
    {
        for (int j = 0; j < cols; j++)
        {
            result[j] ^= matrix[i, j]; // XOR
        }
    }
}

return string.Join("", result);
}

private string DivideBinaryString(string dividend, string g, bool
synd) // Функция деления на полином g
{
    char[] data = dividend.ToCharArray();
    char[] ans = Enumerable.Repeat('0', k).ToArray();
    for (int i = 0; i < data.Length - m; i++)
    {
        if (data[i] == '1')
        {
            // XOR с g, начиная с позиции i
            for (int j = 0; j <= m; j++)
            {
                data[i + j] = (data[i + j] == g[j]) ? '0' : '1';
            }
            ans[i] = '1';
        }
    }

    return synd ? new string(data, data.Length - m, m) : new
string(ans);
}

```

					ТКСВИ.140000.000 ПЗ	Лист
						32
Изм.	Лист	№ докум.	Подпись			


```

public static int GetBchT(string g, int n) // Функция нахождения s
{
    // Словарь: (n, k) -> t
    var bchTable = new Dictionary<(int n, int k), int>
    {
        // n = 7
        {(7, 4), 1},
        {(7, 3), 2},
        // n = 15
        {(15, 11), 1},
        {(15, 7), 2},
        {(15, 5), 3},
        // n = 31
        {(31, 26), 1},
        {(31, 21), 2},
        {(31, 16), 3},
        {(31, 11), 5},
        {(31, 6), 7}
    };

    g = g.Trim();
    if (string.IsNullOrEmpty(g) || !g.All(c => c == '0' || c == '1'))
        return 0;

    int degree = g.Length - 1;
    if (degree <= 0) return 0;
    int firstOne = g.IndexOf('1');
    if (firstOne == -1) return 0;
    degree = g.Length - firstOne - 1;
    int k = n - degree;
    var key = (n, k);
    return bchTable.TryGetValue(key, out int t) ? t : -1;
}

```

					ТКСВИ.140000.000 ПЗ	Лист
						33
Изм.	Лист	№ докум.	Подпись			